

SES MEO Time-Dependent Graph Optimisation

Python implementation for the SES MEO satellite network optimisation assignment.

Implemented methods: One-snapshot Dijkstra, Sequential Dynamic Programming (DP), and Joint Temporal Optimisation.

The prototype uses 18 satellites, 3 orbital planes, 2 gateways and 21 time snapshots with a 30-second interval.

1. Requirements

- Python 3.10+
- PuLP
- NumPy

All Python dependencies are listed in **requirements.txt**. No external dataset or ephemeris file is required to run the supplied prototype.

2. Project Structure

```
ME0_One_Shot_Temporal_Optimization/  
■  
■■■ run.py  
■■■ requirements.txt  
■■■ README.md  
■  
■■■ src/  
■   ■■■ __init__.py  
■   ■■■ config.py  
■   ■■■ network_data.py  
■   ■■■ optimization.py  
■   ■■■ main.py  
■   ■■■ Model.py  
■   ■■■ reporting.py  
■  
■■■ docs/  
■   ■■■ MODEL.md  
■  
■■■ data/  
■  
■■■ results/
```

run.py — Entry point used to run the complete experiment.

src/config.py — Scenario parameters such as number of satellites, time snapshots, demand and handover cost.

src/network_data.py — Generates topology, link characteristics and time-dependent availability.

src/optimization.py — Contains the optimisation algorithms.

src/main.py — Runs the complete experiment and compares the three approaches.

src/reporting.py — Route extraction and reporting utilities.

docs/MODEL.md — Mathematical description of the optimisation model.

results/ — Generated results/cache files.

3. Installation

macOS / Linux

```
cd ME0_One_Shot_Temporal_Optimization  
  
python3 -m venv .venv  
source .venv/bin/activate  
  
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

Windows

```
cd MEO_One_Shot_Temporal_Optimization

python -m venv .venv
.venv\Scripts\activate

python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

4. Run the Experiment

From the project root directory, run:

```
python run.py
```

This is the recommended way to execute the project. Do not run individual files from inside the **src/** directory.

```
Network generation
↓
Time-dependent feasibility
↓
One-snapshot Dijkstra
↓
Sequential Dynamic Programming
↓
Joint temporal optimisation
↓
Comparison of results
```

5. Expected Output

The program prints the optimisation results to the terminal. The final section is **TEMPORAL OPTIMISATION COMPARISON** and contains results for all three methods.

For each method, the following information is reported:

- Total routing latency
- Number of handovers
- Handover cost
- Total objective
- Serving-satellite sequence

The serving sequence is displayed in the form:

```
S2 -> S5 -> S4 -> S18 -> ...
```

A final summary table compares the three methods.

6. Example Execution

```
$ python run.py
```

```
...
```

```
=====
TEMPORAL OPTIMISATION COMPARISON
=====
```

```
Method 1: One-Snapshot Dijkstra
```

```
-----
Total routing latency : ...
Handover count       : ...
Handover cost        : ...
Total objective       : ...
Serving sequence     : S2 -> S5 -> S4 -> ...
```

```
Method 2: Sequential Dynamic Programming
```

```
-----
Feasible              : True
Total routing latency : ...
```

```
Handover count      : ...
Handover cost       : ...
Total objective     : ...
Serving sequence    : S2 -> S5 -> S4 -> ...
```

Method 3: Joint Temporal Optimisation

```
-----
Feasible            : True
Total routing latency : ...
Handover count      : ...
Handover cost       : ...
Total objective     : ...
Serving sequence    : S2 -> S5 -> S4 -> ...
```

The exact numerical values depend on the current configuration and generated scenario.

7. Scenario Configuration

The main scenario parameters are defined in **src/config.py**.

```
Satellites          : 18
Orbital planes      : 3
Satellites/plane    : 6
Gateways            : 2
Time snapshots      : 21
Time step           : 30 s
Demand              : 350 Mbps
Handover cost       : 8
```

The time snapshots are **t0**, **t1**, ..., **t20**, corresponding to 0 s, 30 s, ..., 600 s.

8. Changing the Scenario

To experiment with the model, edit **src/config.py**.

For example, to change the traffic demand:

```
DEMAND = 400.0
```

To change the handover penalty:

```
HANDOVER_COST = 12.0
```

Then run again:

```
python run.py
```

9. Re-running After Changes

The project may store generated feasibility information in:

```
results/stage1_feasibility.pkl
```

If the network generation or configuration is changed, regenerate the results rather than relying on an old cached feasibility file.

The recommended setting is:

```
REUSE_STORED_FEASIBILITY = False
```

Alternatively, remove the cached file before running.

macOS / Linux

```
rm -f results/stage1_feasibility.pkl
```

Windows PowerShell

```
Remove-Item results/stage1_feasibility.pkl
```

```
python run.py
```

10. Understanding the Three Methods

Method	Description
One-snapshot Dijkstra	Each time snapshot is solved independently. The lowest-cost feasible route is selected at each snapshot,
Sequential DP	Candidate routes are generated for each snapshot and Dynamic Programming selects a temporally consis
Joint Temporal Optimisation	All time snapshots are considered jointly in one temporal optimisation problem, including the handover per

The three methods are compared using the same total-cost definition:

$$\text{Total cost} = \text{routing cost} + \text{handover cost}$$

11. Network Data

The current prototype uses a deterministic synthetic time-varying availability model. This means that the prototype does not require external MEO ephemeris data to run.

The network-data generation is implemented in **src/network_data.py**. The optimisation code receives the resulting time-dependent network information and does not depend directly on how the availability data were generated.

This separation allows the synthetic availability model to be replaced later by an ephemeris/geometry-based model without changing the overall optimisation architecture.

12. Results

Generated files are stored under **results/**. The terminal output provides the main comparison between the three methods.

The important quantities to compare are:

- Routing latency
- Handover count
- Handover cost
- Total objective
- Serving-satellite sequence

13. Quick Start for Reviewers

If you only want to run the assignment:

```
cd MEO_One_Shot_Temporal_Optimization
python -m pip install -r requirements.txt
python run.py
```

Then look at the final **TEMPORAL OPTIMISATION COMPARISON** section in the terminal.

14. Code Walkthrough

If reviewing the implementation, the recommended order is:

1. src/config.py
2. src/network_data.py
3. src/optimization.py
4. src/main.py
5. src/reporting.py
6. docs/MODEL.md

main.py shows how the complete experiment is executed. **network_data.py** shows how the time-dependent network is constructed. **optimization.py** contains the optimisation implementations. **docs/MODEL.md** provides the mathematical formulation.

15. Notes

- The supplied scenario is intentionally small so that all three approaches can be compared on the same problem.
- The network availability is synthetic and deterministic.
- No external data download is required.
- Re-running the same configuration produces reproducible network data.
- The project is intended to demonstrate the optimisation methodology rather than provide a full physical MEO orbital simulator.